

Coffee Code — Sistema Integral de Administración de Cafetería

Contexto del proyecto

Proyecto académico de Ingeniería en Sistemas Computacionales (UPQ). Equipo: Martínez Silvestre Coral, Jiménez Vargas Diego, Ledesma Ledesma Emiliano. Asesor: ISC Iván Isay Guerra López.

Este es el **Entregable del Primer Parcial pasado a código funcional**. El análisis (requerimientos, casos de uso, diagramas de secuencia, diccionario de datos, modelo relacional 2FN) ya está cerrado y se documenta abajo. La tarea actual es **construir el sistema real**, no rediseñar el análisis.

Problemática que resuelve

Coffee Code es una cafetería que opera sin sistema digital integrado. Pedidos, comunicación con cocina, cobros e inventario se manejan manualmente, lo que genera errores de comunicación entre roles, tiempos de respuesta altos y descontrol de inventario. Se necesita atender tres roles operativos (mesero, cocina, caja) desde dispositivos móviles en tiempo real, más un panel administrativo web para usuarios/roles y estadísticas.

Arquitectura (desacoplada, 3 componentes + DB única)

Componente	Tecnología	Rol
API Central	FastAPI + Python	Núcleo del sistema: lógica de negocio, JWT, WebSockets. Única capa que toca PostgreSQL (vía SQLAlchemy ORM).
Cliente Móvil	React Native + Expo	App única con navegación por rol: Mesero, Caja, Cocina.
Cliente Web Admin	Flask + Python	Panel exclusivo admin: usuarios/roles, estadísticas, exportación PDF/XLSX. Flask consume la API FastAPI, no accede directo a la DB.
Base de datos	PostgreSQL	Única fuente de verdad. Relación mesas 1:N pedidos (modelo de comandas).

Reglas de arquitectura que deben respetarse en la implementación:

- Toda la lógica de negocio vive en FastAPI. Flask y React Native son clientes.
- Autenticación centralizada vía JWT (`user_id`, `rol`, `exp: 24h`), bcrypt para contraseñas.
- Notificaciones en tiempo real vía WebSockets (cocina→mesero, caja→cocina), objetivo < 2 segundos.

- Descuento de inventario es **atómico**: cambio de estado de pedido a "Listo"
 - descuento de suministros en una sola transacción (rollback si falla).
- Autorización por rol en cada endpoint protegido (middleware JWT + rol).

Diccionario de datos (modelo 2FN, ya validado — implementar tal cual)

```

ROLES(id PK, nombre, descripcion)
USUARIOS(id PK, nombre, apellido_paterno, apellido_materno NULL,
          correo_electronico UNIQUE, password_hash, activo BOOL default true,
          fecha_creacion, id_rol FK->ROLES)
ESTATUS_MESAS(id PK, nombre)          -- Libre, Ocupada, Reservada
MESAS(id PK, numero_mesa UNIQUE, capacidad, activo BOOL, id_estatus FK->ESTATUS_MESAS)
ESTATUS_PEDIDOS(id PK, nombre)        -- Pendiente, En preparación, Listo,
Entregado, Cancelado
ESTATUS_COCINA(id PK, nombre)         -- Pendiente, En preparación, Listo
(por ítem)
PEDIDOS(id PK, fecha, total DECIMAL NULL hasta cerrarse,
          id_mesa FK->MESAS, id_usuario FK->USUARIOS, id_estatus FK->ESTATUS_PEDIDOS)
DETALLE_PEDIDOS(id PK, cantidad >=1, especificaciones TEXT NULL,
precio_unitario,
               id_producto FK->PRODUCTOS, id_pedido FK->PEDIDOS, id_estatus
FK->ESTATUS_COCINA)
CATEGORIAS(id PK, nombre, descripcion NULL, activo BOOL)
PRODUCTOS(id PK, nombre, descripcion TEXT NULL, precio_venta, disponible BOOL,
          activo BOOL, id_categoria FK->CATEGORIAS)
INGREDIENTES(id PK, nombre, unidad, stock_actual DECIMAL default 0,
              stock_minimo DECIMAL, costo_unitario, activo BOOL)
RECETAS(id_producto FK->PRODUCTOS, id_ingredientes FK->INGREDIENTES,    -- PK
compuesta
        cantidad_requerida DECIMAL)
METODOS_PAGO(id PK, nombre)           -- Efectivo, Tarjeta débito/crédito,
Transferencia
TICKETS(id PK, subtotal, iva, total, fecha_emision,
        id_pedido FK->PEDIDOS [1:1], id_usuario FK->USUARIOS)
PAGOS(id PK, monto_recibido, cambio, id_ticket FK->TICKETS [1:1], id_metodo FK->METODOS_PAGO)
GASTOS(id PK, monto, concepto, fecha_gasto, id_usuario FK->USUARIOS)

```

Cardinalidades clave: Mesas 1:N Pedidos · Pedidos 1:N Detalle_pedidos ·
 Productos N:M Ingredientes resuelto vía RECETAS · Pedidos 1:1 Tickets ·
 Tickets 1:1 Pagos.

Endpoints de la API Central (FastAPI) — contrato a implementar

Auth:

- `POST /auth/login` → {email, password} → {access_token, rol} (JWT 24h)
- Middleware de validación de JWT + rol en todos los endpoints protegidos

Mesas / Mesero:

- `GET /mesas` → lista de mesas con estatus
- `POST /pedidos` → {mesa_id, usuario_id, items[]} → crea pedido + detalle (transacción)
- `GET /pedidos/{id}` / `GET /pedidos?estado=`
- `PUT /pedidos/{id}/estado` → cambia estado (incluye "Entregado" que libera mesa si no hay más pedidos activos)

Caja:

- `POST /ventas` → {pedido_id, metodo_pago, monto} → ticket + pago, pasa pedido a "Pendiente" (cola cocina), notifica WS
- `POST /gastos` → {descripcion, monto, categoria}
- `POST /compras` → {ingrediente_id, cantidad, monto} → gasto + incrementa stock
- `GET /caja/resumen` → ventas, gastos, ganancia neta del periodo
- `PUT /pedidos/{id}/estado` → Cancelado (si pago falla)

Cocina:

- `GET /productos`, `POST /productos`, `PUT/DELETE /productos/{id}`
- `GET /ingredientes`, `POST /ingredientes`, `PUT /ingredientes/{id}/stock`
- `POST /producto_ingrediente` (receta) → {producto_id, ingrediente_id, cantidad}
- `GET /pedidos?estado=Pendiente` (cola FIFO por fecha)
- `PUT /pedidos/{id}/estado` → Pendiente→En preparación→Listo (descuento atómico de receta al marcar "Listo" + alerta si stock < stock_minimo) + WS push a mesero

Admin / Reportes (consumidos por Flask):

- `GET /api/usuarios`, `POST /api/usuarios`, `PUT /api/usuarios/{id}` (incl. rol y desactivar)
- `GET /api/reportes?desde=&hasta=` → ventas, gastos, ganancia, top productos (agregado)

WebSockets:

- Canal por pedido / por rol para `nuevo_pedido`, `pedido_listo`, `pedido_activado`

Reglas de negocio no funcionales obligatorias

- Login < 2s. Mesas < 1.5s. Notificaciones WS < 2s. Reportes < 3s (rango ≤ 6 meses).
- No se permite enviar pedido vacío (mínimo 1 ítem).
- No se permiten pagos duplicados (validación de transacción única).
- Descuento de stock al marcar "Listo" es atómico (BEGIN/COMMIT, rollback en fallo).
- Contraseñas con bcrypt + salt. JWT en `Authorization: Bearer {token}`.
- Roles válidos: Mesero, Cajero, Cocinero, Administrador. Acceso a módulo restringido por rol vía middleware (no solo en el frontend).

Alcance del entregable actual (lo que se debe construir ahora)

1. **API funcional completa** con toda la lógica de negocio descrita arriba, lista para ser consumida por web y móvil.
2. **Frontend móvil** (React Native + Expo) consumiendo la API: módulos Mesero, Caja y Cocina con navegación por rol.
3. **Web funcional** (Flask) para: gestión de usuarios y roles + módulo de estadísticas (dashboard, reportes, exportación PDF/XLSX) — consumiendo la API, sin lógica de negocio propia.
4. **Colección de Postman** que pruebe los endpoints principales de Cocina, Caja y Mesero (no de Admin, ese se prueba vía la web).

Convenciones de trabajo

- Español para nombres de tablas, campos, mensajes de error y commits. Inglés está bien para nombres de variables/funciones internas si el equipo lo prefiere, pero mantener consistencia con el diccionario de datos (los nombres de columnas en español, tal cual están arriba).
- Migraciones de DB versionadas (Alembic) desde el día 1, no `create_all` silencioso en producción.
- Variables de entorno (`.env`) para `DB_URL`, `JWT_SECRET`, etc. — nunca hardcodeadas.
- Cada endpoint nuevo debe acompañarse de su request de Postman.